Державний університет «Житомирська політехніка»
Факультет інформаційно-комп’ютерних технологій
Кафедра комп’ютерної інженерії

ПОЯСНЮВАЛЬНА ЗАПИСКА

до лабораторної роботи № 3
з дисципліни «Програмування»

Одновимірні та багатовимірні масиви, обробка символьних рядків у мові С та робота з гілками в Git

Індивідуальний варіант 1

Виконав студент 1 курсу
групи КІ-26-1
Безвершук Ростислав

Перевірив Лисенко М. С.
Посада: Старший викладач кафедри комп'ютерної інженерії та штучного інтелекту
Науковий ступінь: Доктор філософії з інженерії програмного забезпечення

Житомир 2026

1 Мета роботи та індивідуальні завдання

Мета роботи - реалізувати обробку одновимірного масиву та двовимірної матриці, навчитися вручну сканувати символьний рядок до нуль-термінатора, а також відпрацювати створення та злиття гілок Git.

Мій номер у списку - 1. Варіант = ((1 − 1) mod 15) + 1 = 1. Використовую GCC із прапорцями -Wall та -Wextra, термінал і Git.

1.1 Одновимірний масив

Знайти мінімальний за модулем елемент масиву та суму елементів, розташованих після першого від’ємного. Масив int a[SIZE] має 15 елементів; SIZE визначено через #define. Звичайний запуск заповнює його значеннями rand() % 101 − 50 після srand(time(NULL)).

Початковий мінімум дорівнює a[0]. За однакових модулів зберігаю перший елемент. Індекс першого від’ємного спочатку дорівнює −1. Після його знаходження додаю лише наступні елементи. Якщо від’ємних немає, повідомляю про це; якщо перший від’ємний останній, сума порожньої частини дорівнює 0. Індекси починаються з 0; масив не змінюється.

1.2 Двовимірна матриця

Для кожної колонки матриці знайти мінімальний елемент та поміняти місцями перший і останній рядки. Використовую прямокутну матрицю int a[4][5]. Вектор мінімумів початково копіюю з першого рядка. У вкладених циклах зовнішній індекс перебирає рядки, внутрішній - стовпці. Потім виконую обмін крайніх рядків через одну тимчасову змінну.

1.3 Символьний рядок

Підрахувати загальну кількість слів у введеному тексті; слова розділені одним або кількома пробілами. Буфер char str[256] вміщує 255 байтів тексту та байт \0. Використовую fgets і ручний прохід до \0. Прапорець in_word позначає перебування всередині слова; лічильник збільшую лише на переході від пробілу або початку рядка до непробільного символу. Функції <string.h> у task3.c не використовуються.

2 Лістинг task1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define SIZE 15

static void print_array(const int a[SIZE])
{
   for (int i = 0; i < SIZE; ++i) {
      printf("%4d", a[i]);
   }
   putchar('\n');
}

static void analyze(const int a[SIZE])
{
   int min_value = a[0];
   int min_index = 0;
   int first_negative = -1;
   int sum = 0;
   for (int i = 0; i < SIZE; ++i) {
      if (abs(a[i]) < abs(min_value)) {
         min_value = a[i];
         min_index = i;
      }
      if (first_negative >= 0) {
         sum += a[i];
      } else if (a[i] < 0) {
         first_negative = i;
      }
   }
   printf("Minimum by absolute value: %d; index: %d\n",
         min_value, min_index);
   if (first_negative < 0) {
      puts("No negative element; sum is not calculated.");
   } else {
      printf("First negative index: %d\n", first_negative);
      printf("Sum after first negative: %d\n", sum);
   }
}
```

2 Лістинг task1.c продовження

```
int main(int argc, char *argv[])
{
    int a[SIZE];
    if (argc == 2 && strcmp(argv[1], "--demo") == 0) {
        /* Reproducible test; the ordinary run uses rand(). */
        const int demo[SIZE] = {
            12, 7, -8, 4, -3, 0, 15, -2, 9, 6, -1, 5, -4, 11, 2
        };
        for (int i = 0; i < SIZE; ++i) {
            a[i] = demo[i];
        }
        puts("Mode: fixed demonstration");
    } else if (argc == 1) {
        srand((unsigned int)time(NULL));
        for (int i = 0; i < SIZE; ++i) {
            a[i] = rand() % 101 - 50;
        }
        puts("Mode: random values in [-50; 50]");
    } else {
        fprintf(stderr, "Usage: %s [--demo]\n", argv[0]);
        return 1;
    }
    printf("Original array: ");
    print_array(a);
    analyze(a);
    printf("Array after analysis: ");
    print_array(a);  /* This variant does not change the array. */
    return 0;
}
```

Компіляція та запуск

```
gcc -Wall -Wextra task1.c -o task1
./task1
./task1 --demo
```

Режим --demo додано для відтворюваного тесту. Він використовує фіксовані 15 чисел і ту саму функцію analyze, що й звичайний запуск. У звичайному режимі масив генерується псевдовипадково; результати різних запусків можуть відрізнятися.

3 Повний лістинг task2.c

```
#include <stdio.h>

#define ROWS 4
#define COLS 5

static void print_matrix(int a[ROWS][COLS])
{
   for (int i = 0; i < ROWS; ++i) {
      for (int j = 0; j < COLS; ++j) {
         printf("%6d", a[i][j]);
      }
      putchar('\n');
   }
}

static void column_minima(int a[ROWS][COLS], int mins[COLS])
{
   for (int j = 0; j < COLS; ++j) {
      mins[j] = a[0][j];
   }
   /* Rows outside, columns inside: sequential row-major access. */
   for (int i = 1; i < ROWS; ++i) {
      for (int j = 0; j < COLS; ++j) {
         if (a[i][j] < mins[j]) {
            mins[j] = a[i][j];
         }
      }
   }
}

static void swap_outer_rows(int a[ROWS][COLS])
{
   for (int j = 0; j < COLS; ++j) {
      int temp = a[0][j];
      a[0][j] = a[ROWS - 1][j];
      a[ROWS - 1][j] = temp;
   }
}

int main(void)
{
   int a[ROWS][COLS] = {
      { 12, -3,  7,  0,  9},
      {  4,  8, -6, 11,  2},
      { -5,  1, 10, -4,  6},
      {  3, -9,  5,  8, -2}
   };
   int mins[COLS];
   puts("Original matrix:");
   print_matrix(a);
   column_minima(a, mins);
   printf("Column minima: ");
   for (int j = 0; j < COLS; ++j) {
      printf("%6d", mins[j]);
   }
   putchar('\n');
   swap_outer_rows(a);
   puts("After swapping first and last rows:");
   print_matrix(a);
   return 0;
}
```

4 Повний лістинг task3.c

```
#include <stdio.h>

#define MAX_TEXT 255

static int count_words(const char str[])
{
    int count = 0;
    int in_word = 0;
    for (int i = 0; str[i] != '\0'; ++i) {
        if (str[i] == ' ') {
            in_word = 0;
        } else if (!in_word) {
            ++count;
            in_word = 1;
        }
    }
    return count;
}

int main(void)
{
    char str[MAX_TEXT + 1];  /* 255 text bytes plus '\0'. */
    printf("Enter text (up to %d bytes): ", MAX_TEXT);
    if (fgets(str, sizeof(str), stdin) == NULL) {
        fputs("Error: no input line.\n", stderr);
        return 1;
    }
    int length = 0;
    while (str[length] != '\0' && str[length] != '\n') {
        ++length;
    }
    if (str[length] == '\n') {
        str[length] = '\0';
    } else if (length == MAX_TEXT) {
        int ch = getchar();
        if (ch != '\n' && ch != EOF) {
            while ((ch = getchar()) != '\n' && ch != EOF) {
                /* Discard the remainder of an overlong line. */
            }
            fputs("Error: text exceeds 255 bytes.\n", stderr);
            return 1;
        }
    }
    /* Also remove CR if the input uses CRLF line endings. */
    if (length > 0 && str[length - 1] == '\r') {
        str[--length] = '\0';
    }
    printf("Text: [%s]\n", str);
    printf("Word count: %d\n", count_words(str));
    return 0;
}
```

Компіляція двох останніх програм

```
gcc -Wall -Wextra task2.c -o task2
gcc -Wall -Wextra task3.c -o task3
```

5 Перевірка масиву та матриці

У звичайному запуску ./task1 отримую 15 цілих чисел у діапазоні [−50; 50]. Початковий масив і масив після аналізу однакові, оскільки варіант передбачає лише обчислення.

```
hallt@DragonWing ~/p/p/lab03 (lab03)> ./task1
Mode: random values in [-50; 50]
Original array:   -8 -20  12  37 -36  49 -21  14  23 -41 -44 -26  -9  39 -40
Minimum by absolute value: -8; index: 0
First negative index: 0
Sum after first negative: -63
Array after analysis:   -8 -20  12  37 -36  49 -21  14  23 -41 -44 -26  -9  39 -40
hallt@DragonWing ~/p/p/lab03 (lab03)>
```

Рисунок 1 - Випадковий масив і результати його обробки

Для ./task1 --demo використовую масив: 12, 7, −8, 4, −3, 0, 15, −2, 9, 6, −1, 5, −4, 11, 2. Мінімальний за модулем елемент - 0 з індексом 5. Перший від'ємний - −8 з індексом 2; сума елементів з індексами 3…14 дорівнює 42.

```
hallt@DragonWing ~/p/p/lab03 (lab03)> ./task1 --demo
Mode: fixed demonstration
Original array:   12   7  -8   4  -3   0  15  -2   9   6  -1   5  -4  11   2
Minimum by absolute value: 0; index: 5
First negative index: 2
Sum after first negative: 42
Array after analysis:   12   7  -8   4  -3   0  15  -2   9   6  -1   5  -4  11   2
hallt@DragonWing ~/p/p/lab03 (lab03)>
```

Рисунок 2 - Відтворюваний тест масиву в режимі --demo

У task2.c початкова матриця містить рядки (12, −3, 7, 0, 9), (4, 8, −6, 11, 2), (−5, 1, 10, −4, 6), (3, −9, 5, 8, −2). Мінімуми стовпців: (−5, −9, −6, −4, −2). Після обміну першим стає четвертий рядок, четвертим - перший; два середні зберігаються.

```
hallt@DragonWing ~/p/p/lab03 (lab03)> ./task2
Original matrix:
    12    -3     7     0     9
     4     8    -6    11     2
    -5     1    10    -4     6
     3    -9     5     8    -2
Column minima:     -5    -9    -6    -4    -2
After swapping first and last rows:
     3    -9     5     8    -2
     4     8    -6    11     2
    -5     1    10    -4     6
    12    -3     7     0     9
hallt@DragonWing ~/p/p/lab03 (lab03)>
```

Рисунок 3 - Мінімуми стовпців та обмін крайніх рядків матриці

6 Перевірка обробки тексту

Слова визначаю як максимальні послідовності байтів, відмінних від звичайного пробілу. Початкові, кінцеві та повторні пробіли не утворюють додаткових слів. Розділові знаки всередині такої послідовності належать слову. Табуляція не є роздільником у цьому варіанті.

Введений рядок	Очікувано слів
Hello world	2
one two three	3
word	1
порожній рядок	0
п’ять пробілів	0
one\ttwo (табуляція)	1
Привіт світе	2

Під час зчитування видаляю кінцевий \n та, за наявності, \r. Символ \0 залишається коректним. Для UTF-8 обмеження 255 стосується байтів, а не кількості літер; підрахунок слів з ASCII-пробілами працює і для українського тексту.

```
hallt@DragonWing:~/projects/programming-sem1/lab03$ for text in 'Hello world' '  one   two  three  ' 'word' '' '     '; do
    printf '%s\n' "$text" | ./task3
done
Enter text (up to 255 bytes): Text: [Hello world]
Word count: 2
Enter text (up to 255 bytes): Text: [  one   two  three  ]
Word count: 3
Enter text (up to 255 bytes): Text: [word]
Word count: 1
Enter text (up to 255 bytes): Text: []
Word count: 0
Enter text (up to 255 bytes): Text: [     ]
Word count: 0
hallt@DragonWing:~/projects/programming-sem1/lab03$
```

Рисунок 4 - Підрахунок слів за різної кількості пробілів та в порожньому рядку

Рядок довжиною рівно 255 байтів приймається. Якщо введено понад 255 байтів, програма дочитує залишок рядка, виводить Error: text exceeds 255 bytes. і завершується з кодом 1. Порожній рядок із Enter є допустимим; відсутність введення до EOF дає окрему помилку.

```
hallt@DragonWing:~/projects/programming-sem1/lab03$ printf '%0256d\n' 0 | tr '0' 'a' | ./task3
echo "Exit code: $?"
Error: text exceeds 255 bytes.
Enter text (up to 255 bytes): Exit code: 1
hallt@DragonWing:~/projects/programming-sem1/lab03$
```

Рисунок 5 - Відхилення тексту завдовжки понад 255 байтів

7 Організація пам’яті та складність алгоритмів

Одновимірний масив займає SIZE · sizeof(int) байтів. Для 15 елементів і sizeof(int) = 4 це 60 байтів. Аналіз виконується за O(SIZE) часу та O(1) додаткової пам’яті. Через діапазон [−50; 50] функція abs не отримує INT_MIN, а модуль суми не перевищує 14 · 50 = 700.

У C елементи матриці розташовані за рядками. Адреса A[i][j] дорівнює адресі A[0][0] плюс (i · COLS + j) · sizeof(int). Для матриці 4 × 5 та 4-байтового int потрібно 80 байтів. COLS визначає довжину рядка й крок переходу між рядками.

Під час пошуку мінімумів спочатку послідовно читаю перший рядок, потім усі наступні. Зовнішній цикл перебирає i, внутрішній - j: сусідні звернення до матриці стосуються сусідніх елементів. Для великих матриць це краще використовує кеш-лінії; для нашої малої матриці вимірювання прискорення не проводилося. Обмін рядків проходить двома послідовними потоками пам’яті.

Пошук мінімумів потребує O(ROWS · COLS) часу та O(COLS) пам’яті під вектор mins. Обмін крайніх рядків виконує COLS обмінів, тобто O(COLS) операцій і O(1) допоміжної пам’яті. Перестановка рядків не змінює множину значень жодного стовпця, тому його мінімум зберігається.

Для тексту довжиною L байтів ручний підрахунок слів має складність O(L), а додаткові лічильник і прапорець займають O(1) пам’яті. Буфер виділено за правилом N + 1: 255 байтів тексту та один нуль-термінатор. Без \0 функція printf з форматом %s могла б читати поза межами масиву, що є невизначеною поведінкою.

Усі програми перевірено компіляцією GCC із -std=c11 -Wall -Wextra -Wpedantic -Werror: попереджень і помилок немає. Додатково для функції обробки масиву перевірено 105 наборів, зокрема без від’ємних, із нулями, однаковими модулями та від’ємним останнім елементом; для тексту перевірено граничні довжини й некоректне введення.

8 Робота з гілками Git

Створюю окрему гілку lab03 від main та каталог lab03. Для кожного програмного файла виконую окремий коміт. Після вставлення власних скріншотів зберігаю записку як Report_Lab03.pdf і додаю її до цієї самої гілки.

```
git switch main
git switch -c lab03
mkdir -p lab03
cd lab03
# Copy task1.c, task2.c and task3.c here.
git add task1.c
git commit -m "feat(lab3): implement 1D array operations"
git add task2.c
git commit -m "feat(lab3): implement 2D matrix transformation"
git add task3.c
git commit -m "feat(lab3): implement manual string parsing"
```

Після підготовки PDF завершую роботу такими командами. Параметр --no-ff зберігає окремий коміт злиття навіть за можливості fast-forward; це робить історію гілки наочною.

```
git add Report_Lab03.pdf
git commit -m "docs(lab3): add mandatory engineering PDF report"
cd ..
git switch main
git merge --no-ff lab03 -m "merge: integrate lab03"
git log --graph --oneline --decorate --all -12
git push origin main
git push -u origin lab03
```

У каталозі lab03 репозиторію мають бути task1.c, task2.c, task3.c та Report_Lab03.pdf. Виконувані файли task1, task2, task3 або їхні версії з розширенням .exe не додаю до комітів. HEAD під час роботи в гілці позначає поточну гілку, а вона вказує на її останній коміт.

9 Висновки

Для варіанта 1 реалізовано три програми мовою C. У task1.c знайдено мінімальний за модулем елемент та суму після першого від'ємного. Початковий мінімум узято з нульового елемента, однакові модулі обробляються зі збереженням першої позиції. Окремо враховано відсутність від'ємних і порожню частину після останнього елемента.

У task2.c визначено мінімуми всіх п'яти стовпців прямокутної матриці та поміняно місцями її перший і останній рядки. Обхід організовано за рядками відповідно до Row-Major Order. На контрольному прикладі отримано вектор (−5, −9, −6, −4, −2), а після перестановки середні рядки збережено.

У task3.c підрахунок слів реалізовано вручну за допомогою переходів між станами «поза словом» і «всередині слова». Повторні пробіли, пробіли на краях та порожній рядок обробляються коректно. Використано fgets, запас під нуль-термінатор і відхилення рядків, які перевищують місткість буфера.

Результати контрольних запусків узгоджуються з ручними розрахунками. Вивчено послідовне розміщення масивів у пам'яті, роль кількості стовпців в адресації та вимоги до завершення С-рядків. Для обліку змін використано окрему гілку lab03 та її злиття з main; історія демонструється на рисунку 6. Мету роботи досягнуто.